PERTEMUAN 4 · Praktik terbimbing

Finite Automata: NFA, DFA, Konversi, dan Minimalisasi

Universitas Bale Bandung

Fakultas Teknologi Informasi · Program Studi Teknik Informatika

Mata Kuliah Teknik Kompilasi (MWCA017) · Semester Ganjil · 3 SKS

Dosen: Cecep Suwanda, S.Si., M.Kom.

Capaian Pertemuan

CPMK induk: CPMK 02 — Memahami konsep Otomata Hingga (*Finite Automata*).

Sub-CPMK yang dinilai pada pertemuan ini

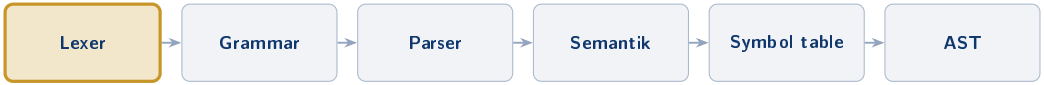
- **Sub-CPMK 2.2:** Mengimplementasikan simulator NFA dan DFA dalam bahasa C untuk mengenali token, dengan berkas kode dan keluaran pengujian sebagai bukti.
- **Sub-CPMK 2.3:** Membuat tabel konversi NFA ke DFA dan tabel minimalisasi DFA, serta menguji bahwa keduanya menerima bahasa token yang sama.

Agenda Pertemuan (150 menit)

Menit	Kegiatan
25	Ceramah: DFA, NFA, dan *epsilon closure*
20	Demonstrasi: menelusuri string pada DFA dan NFA
55	Praktik: *subset construction* dan tabel konversi
30	Praktik: minimalisasi dan tabel partisi
20	Simulator C, pengujian, dan pemeriksaan bersama

Porsi praktik 105 menit lebih besar daripada ceramah 25 menit. Setiap mahasiswa mengumpulkan tabel dan simulator pada akhir pertemuan.

Posisi Pertemuan pada Rantai Proyek



- Pertemuan ini memperkuat simpul **lexer**: dari pattern menuju mesin pengenal.
- DFA yang dibangun hari ini menjadi inti lexer tangan pada Pertemuan 5.
- Tabel konversi dan minimalisasi adalah artefak Sub-CPMK 2.2 dan 2.3 yang dinilai pada Praktikum 1.

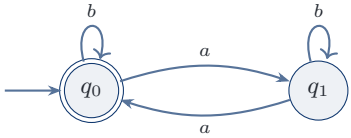
DFA: Definisi

Deterministic Finite Automaton (DFA)

$M = (Q, \Sigma, \delta, q_0, F)$, dengan Q himpunan state berhingga, Σ alfabet, $\delta : Q \times \Sigma \to Q$ fungsi transisi total, q_0 state awal, dan $F \subseteq Q$ himpunan state penerima.

- Untuk setiap pasangan state dan simbol ada tepat satu state tujuan.
- Tidak ada transisi ε dan tidak ada pilihan ganda.
- Simulasi hanya menyimpan satu state saat ini.
- Kompleksitas waktu $O(n)$, ruang $O(1)$ di luar tabel.

Contoh DFA: Jumlah Huruf a Genap



$L = \{w \mid \text{jumlah } a \text{ genap}\}$

State	a	b
$\rightarrow *q_0$	q_1	q_0
q_1	q_0	q_1

Tanda $\rightarrow$ adalah state awal; $*$ menandai state penerima.

Simulasi DFA

Fungsi transisi diperluas

$\hat{\delta}(q, \varepsilon) = q, \quad \hat{\delta}(q, wa) = \delta(\hat{\delta}(q, w), a)$. String w diterima jika $\hat{\delta}(q_0, w) \in F$.

1. Mulai dari q_0.
2. Untuk setiap simbol masukan, pindah ke $\delta(q, a)$.
3. Jika transisi tidak terdefinisi, tolak.
4. Setelah masukan habis, terima jika state berada di F.

Contoh: $abba$ pada DFA sebelumnya berakhir di q_0, sehingga diterima karena jumlah a genap (dua).

Simulator DFA dalam Bahasa C

```c
int run_dfa(const DFA *d, const int *input, int n) {
    int state = d->start;
    for (int i = 0; i < n; i++) {
        state = d->delta[state][input[i]];
        if (state < 0) return 0;
    }
    return d->accept[state];
}
```

Listing 1: Fungsi `run_dfa` untuk simulasi DFA

- `DFA` menyimpan `delta[state][simbol]`, `start`, dan `accept`.
- Tabel transisi dua dimensi membuat simulasi $O(n)$.
- State negatif menandakan transisi tidak terdefinisi.
- Simulator NFA memakai himpunan state dan *epsilon closure*.

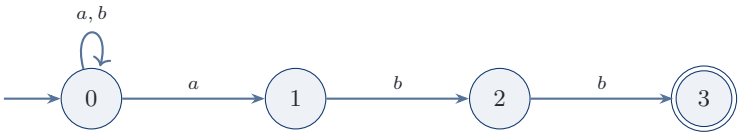
NFA: Definisi

Nondeterministic Finite Automaton (NFA)

$M = (Q, \Sigma, \delta, q_0, F)$, dengan $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \to \mathcal{P}(Q)$. Untuk setiap state dan simbol, NFA dapat memiliki nol, satu, atau beberapa state tujuan.

- Mesin dapat "menebak" langkah yang benar.
- String diterima jika ada setidaknya satu jalur dari q_0 ke state penerima.
- NFA tidak lebih kuat daripada DFA, tetapi lebih mudah disusun dari ekspresi reguler.
- Simulasi menyimpan himpunan state, bukan satu state.

Contoh NFA: String Berakhir dengan `abb`



- NFA menerima $(a \mid b)^*abb$.
- Di state 0, mesin dapat tetap di 0 atau pindah ke 1 saat membaca a.
- Pilihan inilah inti nondeterminisme.

Epsilon Closure

Definisi

ε-closure(S) adalah himpunan state yang dapat dicapai dari S melalui nol atau lebih transisi ε.

- Dihitung dengan penelusuran graf; kompleksitas linear terhadap jumlah transisi.
- Transisi simbol harus mempertimbangkan seluruh state dalam closure.

Contoh

Transisi $q_0 \xrightarrow{\varepsilon} q_1$, $q_1 \xrightarrow{\varepsilon} q_2$, $q_2 \xrightarrow{a} q_3$.
ε-closure($\{q_0\}$) $= \{q_0, q_1, q_2\}$; membaca a mencapai q_3.

Simulasi NFA

1. Mulai dengan ε-closure$(\{q_0\})$.
2. Untuk setiap simbol a, hitung himpunan state tujuan dari semua state saat ini, lalu ambil *epsilon closure*-nya.
3. Jika himpunan state kosong, tolak.
4. Setelah masukan habis, terima jika himpunan state beririsan dengan F.

Kompleksitas

Simulasi NFA sederhana memerlukan $O(n \cdot |Q|^2)$. Untuk lexer, NFA biasanya dikonversi menjadi DFA agar simulasi tetap $O(n)$.

Subset Construction: NFA ke DFA

1. State awal DFA adalah ε-closure($\{q_0\}$).
2. Untuk setiap state DFA S yang belum diproses dan setiap simbol a:
    2. hitung $T = \varepsilon\text{-closure}(\bigcup_{q \in S} \delta(q, a))$;
    2. jika T belum ada, tambahkan sebagai state DFA;
    2. tetapkan $\delta_D(S, a) = T$.
3. State DFA menerima jika $S \cap F \neq \varnothing$.

Jumlah state DFA paling banyak $2^{|Q|}$; dalam praktik banyak himpunan tidak terjangkau sehingga hasilnya lebih kecil.

Contoh Perhitungan *Subset Construction*

- State awal $A = \{0\}$.
- Dari A membaca a: $\text{move}(A, a) = \{0, 1\}$, closure $\{0, 1\}$, jadi $A \xrightarrow{a} B$.
- Dari A membaca b: $\text{move}(A, b) = \{0\}$, jadi $A \xrightarrow{b} A$.
- Dari $B = \{0, 1\}$ membaca a: $\{0, 1\}$, jadi $B \xrightarrow{a} B$.
- Dari B membaca b: $\{0, 2\}$, jadi $B \xrightarrow{b} C$.

Lanjutan

Dari $C = \{0, 2\}$ membaca b menghasilkan $\{0, 3\} = D$; dari D membaca b kembali ke A. State penerima adalah D karena memuat state NFA 3.

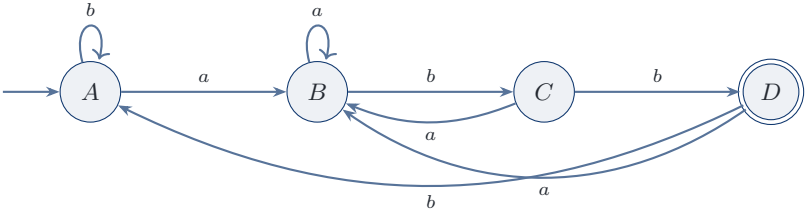
Tabel Konversi NFA ke DFA (Artefak Sub-CPMK 2.3)

State DFA	Himpunan NFA	a	b	Penerima
A	$\{0\}$	B	A	tidak
B	$\{0,1\}$	B	C	tidak
C	$\{0,2\}$	B	D	tidak
D	$\{0,3\}$	B	A	ya

Cara mengisi

Setiap baris dihitung dari NFA asal: hitung move(S, a), ambil *epsilon closure*-nya, lalu beri nama state baru bila himpunan itu belum muncul.

DFA Hasil Konversi



DFA ini menerima bahasa yang sama dengan NFA asal, yaitu seluruh string yang berakhir dengan `abb`.

Minimalisasi DFA: Ekuivalensi State

Ekuivalensi state

Dua state p dan q ekuivalen jika untuk setiap string w, $\hat{\delta}(p, w) \in F$ tepat ketika $\hat{\delta}(q, w) \in F$. State yang tidak ekuivalen disebut terbedakan.

1. Partisi awal: $\{F, Q \setminus F\}$.
2. Kelompokkan state dalam satu grup berdasarkan vektor tujuan transisinya terhadap grup saat ini.
3. Ulangi sampai tidak ada grup yang terpecah.
4. Gabungkan state dalam grup yang sama.

Tabel Minimalisasi DFA (Artefak Sub-CPMK 2.3)

Iterasi	Partisi	State dipisah	Alasan
0	$\{A, B, C\}, \{D\}$	—	State penerima dipisahkan dari bukan penerima
1	$\{A, B\}, \{C\}, \{D\}$	C	$C \xrightarrow{b} D$, sedangkan $A, B \xrightarrow{b}$ ke $\{A, B, C\}$
2	$\{A\}, \{B\}, \{C\}, \{D\}$	A, B	$A \xrightarrow{b} A$, sedangkan $B \xrightarrow{b} C$

Setiap pemisahan disertai alasan berdasarkan transisi. Hasil akhir empat grup berarti DFA sudah minimal.

Konstruksi Thompson: Ekspresi Reguler ke NFA

Ekspresi	Konstruksi
$\varnothing$	State awal dan penerima tanpa jalur
ε	Transisi epsilon dari awal ke penerima
a	Transisi a dari awal ke penerima
$r \mid s$	State awal baru bercabang ke NFA r dan s
rs	State penerima r dihubungkan ke awal s
r^*	Loop epsilon di sekitar NFA r

regex $\rightarrow$ NFA Thompson $\rightarrow$ DFA $\rightarrow$ DFA minimal

Menggabungkan Token dan *Maximal Munch*

- Buat NFA terpisah untuk setiap pattern token.
- Tambahkan state awal baru dengan transisi ε ke setiap NFA.
- State penerima setiap NFA diberi label jenis token.
- Jika beberapa token cocok pada panjang sama, label dengan prioritas tertinggi yang dipilih.

Maximal munch pada DFA

Jalankan DFA selama masih ada transisi, sambil mengingat posisi terakhir di state penerima. Ketika transisi berikutnya tidak ada, lexer kembali ke posisi terakhir dan mengeluarkan token. Pendekatan ini memerlukan *rollback* pada buffer masukan.

Kegiatan Mahasiswa di Kelas: P1-B

Simulator NFA/DFA dan tabel konversi

1. Gambar NFA untuk setiap kategori token pada `spec/tokens.md`.
2. Lakukan *subset construction*; isi tabel konversi.
3. Minimalisasi DFA; isi tabel partisi beserta alasan.
4. Implementasikan simulator NFA dan DFA dalam C.
5. Uji kedua simulator dengan 30 kasus uji dari P1-A.
6. Pastikan NFA, DFA hasil konversi, dan DFA minimal menerima bahasa token yang sama.

Bandingkan hasil dengan alat pembanding laboratorium; catat perbedaan perilaku, tetapi artefak utama tetap simulator C.

Tugas dan Artefak

Artefak Sub-CPMK 2.2 dan 2.3

Simulator NFA dan DFA dalam bahasa C, tabel konversi NFA ke DFA, tabel minimalisasi, serta bukti bahwa ketiganya menerima bahasa token yang sama.

- Tabel konversi lengkap: state DFA, himpunan NFA, transisi setiap simbol, dan status penerima.
- Tabel minimalisasi memuat iterasi, partisi, dan alasan setiap pemisahan.
- Simulator C berjalan dan diuji pada 30 kasus uji P1-A.
- Keputusan terima/tolak NFA, DFA hasil konversi, dan DFA minimal identik pada semua kasus.

P1-B menjadi bagian Praktikum 1 (12%) yang dilaporkan pada Pertemuan 5.

Rangkuman dan Jembatan

- DFA deterministik: satu transisi per state-simbol, mudah disimulasikan dalam $O(n)$.
- NFA dapat memiliki banyak tujuan dan transisi ε; string diterima jika ada satu jalur penerima.
- *Epsilon closure* menjadi inti simulasi NFA dan *subset construction*.
- *Subset construction* mengubah NFA menjadi DFA; minimalisasi menggabungkan state ekuivalen.
- Simulator C, tabel konversi, dan tabel minimalisasi membuktikan bahasa token yang sama.

Pertemuan berikutnya

Pertemuan 5 mengimplementasikan lexer tangan dan lexer Flex. Automata yang dibangun hari ini menjadi mesin pengenal token pada Praktikum 1.